ЗАСОБИ ПРОГРАМУВАННЯ ОБМІНУ ПОВІДОМЛЕННЯМИ

Особливості обміну повідомленнями

- На відміну від засобів паралельного програмування, орієнтованих на спільну пам’ять (семафори, критичні області та монітори), обмін повідомленнями може застосовуватись як на архітектурах з *спільною так і з розподіленою* пам’яттю.
- Обмін повідомленнями передбачає *спочатку синхронізацію* процесів, а *потім передачу інформації* між ними.
- Часто ці дві фази виконання взаємодії процесів поєднуються в *єдиній синтаксичній конструкції*.

Префіксні команди Дійкстри (1)

- Загальну конструкцію і семантику мов програмування, в рамках яких виявилося зручним використовувати засоби паралельного програмування обміну повідомленнями, було запропоновано Е.Дійктрою під назвою *префіксних команд* (guarded commands, GC)
- **GC**. На відміну від традиційних процедурних засобів, що є детермінованими, префіксні команди є недетермінованими.
- Базовий оператор цієї мови має вигляд:

$$B \rightarrow com,$$

- де B (префікс) є логічна умова, істинність якої дозволяє виконання оператора com.
- Програми мовою префіксних команд являють собою вирази, побудовані з базових операторів за допомогою конструкторів альтернативи та циклу.

Префіксні команди Дійкстри (2)

- *Альтернативний оператор* являє собою послідовність базових операторів, відділених один від одного *розділовим знаком* [], що має вид:
    - if $B_1 \rightarrow C_1$
    - [] $B_2 \rightarrow C_2$
    - …
    - [] $B_n \rightarrow C_n$
    - fi
- Семантика його виконання полягає полягає у тому, що якщо жоден з префіксів алтернатив не істинний, то виконується пустий оператор skip, інакше недетерміновано вибирається один з операторів C_i, для яких B_i=<u>true</u>

Префіксні команди Дійкстри (3)

- Циклічний оператор синтаксично нагадує альтернативний:
    - do $B_1 \rightarrow C_1$
    - [] $B_2 \rightarrow C_2$
    - ...
    - [] $B_n \rightarrow C_n$
    - od,
- але порядок його виконання складніший.
- Він задає повторення виконання тіла оператора як альтернативного оператора доти, доки хоча б один з префіксів істинний.

Приклад: алгоритм Евкліда

- Узагальнення поняття програми, пов′язане з недетермінованими префіксними командами, виявилось дуже корисним не тільки для моделей паралельних обчислень, а й взагалі для *теорії програмування*.
- Воно дозволило, по-перше, закласти властивість недетермінованості обчислень в програмування a priori, а по-друге, значно спростити запис алгоритмів.
- Наприклад, алгоритм Евкліда для знаходження найбільшого спільного дільника двох чисел a,b>0 у нотації префіксних команд має вид програми всього в три рядки:
    - do a > b $\rightarrow$ a := a-b;
    - [] b > a $\rightarrow$ b :=b-a;
    - od
- Цей алгоритм можна розглядати також як запис паралельної програми з двома процесами, що синхронізуються умовами a>b і b>a і закінчують роботу, тільки тоді, коли a=b.

Модельна мова CSP

- Ідеї префіксних команд і недетермінованих програм були використані у мовах паралельного програмування засобів обміну повідомленнями.
- **CSP.** Програма на CSP (A. Hoare) являє собою скінченну множину процесів { P1 || P2 .. || PN], що паралельно функціонують і мають адресні простори, що не перетинаються, тобто, процеси *не мають спільної пам'яті*.
- Взаємодія процесів забезпечується командами *прийому і передачі*, що є *парними командами*.
- Тобто, кожній команді передачі з процесу А до процесу В обов'язково мусить бути передбачена команда прийому в процес В з процесу А.
- Так наприклад, оператор P_2 ! X означає передачу повідомлення, тип якого визначається змінною X, у процес P_2 Відповідно, P_1 ? y означає прийом повідомлення з процесу P_1 за адресою y.

Семантика обмінів рандеву

- Передбачається *синхронне виконання* команд передачі та прийому, результат якого семантично еквівалентний оператору присвоювання y:=x.
- Такий спосіб виконання операцій передачі/прийому стали називати *рандеву* (rendezvous). При цьому ніякого буфера не вимагається для обміну повідомленнями.
- Параметрами команд передачі та прийому можуть бути не тільки змінні , а й вирази, зокрема пусті.
- В останньому випадку передача значення пустого виразу означає *просто синхронізацію* — передачу сигналу без обміну даними.

Особливості синтаксису CSP

- Оператори CSP мають за основу префіксні команди Дійкстри з дещо *відмінним* синтаксисом.
- Наприклад, *[] в CSP означає конструктор циклічного повторення.
- Але більшою особливістю CSP є те, що в префіксах можуть використовуватись команди *передачі і прийому*, що надає CSP більшої виразної сили.
- Відповідність прийому і передачі у мові CSP є *симетричною.*
- Наприклад, процес-посередник (буфер на одне місце) для задачі обмеженого буфера можна записати:
  - buf:: *[i : item; Producer ? i $\rightarrow$ Consumer ! i]
  - Producer:: … Buf ! i ….
  - Consumer:: … Buf ? i

Задача про обмежений буфер

- Задача обмеженого буфера (постачальник-споживач) має вигляд:
- PC:: P || B ||C;
- P:: *[p: item: C:: *[c: item
- Gen(p) B?c
- B!p] use(c);]
- B:: busy:integer; busy:=0; b:item;
- *[if busy<n; P?b → Deposit(b); busy:=busy + 1**;** (*розміщення елемента*)
- [] busy>0; C!b → Remove(b); busy:=busy-1; (звільнення елемента*)
-]
- <u>Зауваження</u>. Цей варіант програми не деталізує внутрішню структуру буфера і залишає "за кадром" операції розміщення та звільнення місця в буфері, позначені функціями Deposit(b) та Remove(b).

Мова Ada

- Принцип рандеву застосовується і у мові Ada, хоча в Ada є і спільні змінні.
- Основними структурними одиницями програми Ada є *задачі* (tasks), що можуть викликатись ззовні за допомогою оператора

  accept<ім'я_входу>[<список параметрів>] [do < оператор > end]
- (в квадратні дужки взято фрагменти оператора, що є його опціями).
- На відміну від CSP, в Ada відповідність задачі, що викликає, і задачі, яку викликають – *несиметрична*. Задача, яку викликають, "не знає", хто є ініціатором виклику.
- Крім того, рандеву не зводиться тільки до передачі параметрів, а й може передбачати *повернення результату*.
- Оператор accept більше нагадує процедуру, а виклик входу – віддалений виклик процедур (remote procedure call) в розподілених системах.

Оператор select

- Оператор select вживається в Ada для недетермінованого вибору входу:
- select
- when B1 $\rightarrow$ accept <A1>do S_1 end
- or
- when B2$\rightarrow$<A2>do S_2 end
- or …
- else S_n
- end select
- Семантика оператора select полягає у:
  - 1) визначенні відкритих альтернатив, тобто таких, що мають істинні префікси B_i;
  - 2) визначенні тих операторів accept, що чекають на рандеву;
  - 3)якщо таких більше одного, то і у визначенні одного з них для завершення рандеву;
  - 4)очікуванні на виклик входу, якщо else не передбачено і жоден з процесів не чекає на вхід A_i.

Приклад – задача обмеженого буфера

- Прикладом може бути задача обмеженого буфера.

- b, item; busy=0: integer;
- select
- when busy< n $\rightarrow$ accept PUT(b) do Deposit(b) end;
- busy:= busy+1;
- or when busy>0 $\rightarrow$ accept GET(b) do Remove(b) end;
- busy:= busy-1;
- end select;

Три способи взаємодій паралельних процесів

- Віддалений виклик процедур називається ще викликом типу клієнт-сервер, при якому серверна частина, виконуючи запит клієнта, виконує процедуру і повертає, якщо це передбачено, результат клієнту у зворотному напрямку.
- Таким чином, існує три способи взаємодій паралельних процесів:
- через спільну пам’ять (Java, C# та ін.)
- (симетричним) обміном повідомленнями (CSP, MPI)
- (несиметричним) віддаленим викликом процедур (Ada).